System Design and Observability Playbook (Senior Backend)

1) How to Answer System Design in Interviews

Use this sequence:

- Clarify requirements.
- Define SLIs/SLOs and traffic profile.
- Propose high-level architecture.
- Deep dive into data model and critical flows.
- Discuss scaling, failure handling, and security.
- Add observability and operations.
- Explain trade-offs and alternatives.

2) Canonical Architecture Components

- API gateway/load balancer.
- stateless application tier.
- primary DB + read scaling strategy.
- cache layer.
- async messaging/event stream.
- search/analytics side stores as needed.
- observability and alerting stack.

3) Capacity Planning Basics

Estimate:

- requests per second.
- payload size and bandwidth.
- read/write ratio.
- storage growth and retention.
- latency budget split across tiers.

Use back-of-envelope calculations and state assumptions.

4) Scalability Patterns

- Horizontal scaling for stateless services.
- Read replicas and partitioning.
- Queue-based buffering for spikes.
- Caching with invalidation discipline.

5) Reliability Patterns

- Timeout budgets.
- Retry with exponential backoff and jitter.
- Circuit breakers.
- Bulkhead isolation.
- Idempotency and deduplication.

- Graceful degradation.

6) Data Consistency Patterns

- Strong consistency where business critical.
- Eventual consistency for scalable asynchronous workflows.
- Saga/orchestration/choreography for distributed transactions.
- Outbox pattern for reliable event publication.

7) Observability Deep Dive

7.1 Metrics

Golden signals:

- latency.
- traffic.
- errors.
- saturation.

Service-level metrics:

- p50/p95/p99 latency.
- success/error rate by endpoint.
- queue lag and processing delay.
- DB query latency and lock waits.

7.2 Logs

- structured logs with request IDs and trace IDs.
- avoid noisy high-cardinality unbounded fields.
- define retention and privacy policy.

7.3 Tracing

- end-to-end request path visualization.
- identify tail-latency and dependency hot spots.

8) Alerting and On-Call Quality

Good alerts:

- actionable.
- low-noise.
- SLO and symptom focused.

Examples:

- sustained p95 breach.
- error budget burn rate alert.
- queue lag beyond recovery threshold.

9) Incident Management Framework

- Detect quickly.
- Mitigate first, investigate second.
- Communicate clearly.

- Conduct blameless postmortem.
- Track actions to closure.

10) Deployment Safety in System Design Answers

- feature flags.
- canary rollouts.
- blue/green for risky migrations.
- automatic rollback on key signal degradation.

11) Security in Architecture Interviews

- authentication and authorization boundaries.
- data encryption at rest and in transit.
- secret lifecycle management.
- network segmentation.
- auditing and compliance considerations.

12) Example Design: Order Management Platform

Flow:

- API receives create order request.
- write order in Postgres within transaction.
- write outbox event in same transaction.
- outbox relay publishes event to Kafka.
- inventory and payment consumers process asynchronously.
- cache and read model updated for fast reads.

Failure controls:

- idempotency keys for create order.
- retries with DLQ for failed consumers.
- compensating actions for failed payment path.

13) What Senior Interviewers Want to Hear

- explicit constraints and assumptions.
- realistic bottlenecks and mitigation.
- safe migration plan, not just final architecture.
- operational ownership (runbooks, SLOs, incident response).

14) Story Templates (Fill with Your Real Numbers)

Template A: Latency reduction

- Baseline p95:
- Bottleneck found:
- Change made:
- New p95:
- Guardrails added:

Template B: Reliability improvement

- Incident pattern:

- Root causes:
- Controls introduced:
- Incident rate before/after:

Template C: Scale project

- Traffic increase:
- Architecture changes:
- Cost and performance impact:
- Rollback strategy used:

15) Rapid-Fire Design Questions

1. How do you handle sudden 10x traffic burst?

- load shedding, queue buffering, autoscaling, cache hardening, graceful degradation.

2. How do you design for exactly-once effects?

- idempotent writes + dedup keys + transactional boundaries.

3. How do you choose SQL vs NoSQL?

- consistency, query patterns, scale model, operational needs.

4. How do you run safe schema migrations?

- expand-contract, phased rollout, compatibility windows.

5. How do you avoid alert fatigue?

- SLO-based alerts, dedup, clear ownership and runbooks.

16) Concrete Design Snippets and Explanations

16.1 Idempotent Create API with Unique Key

```
CREATE TABLE idempotency_keys (
    key text PRIMARY KEY,
    response_json jsonb NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now()
);
```

```
if key exists:
    return stored response
else:
    execute business transaction
    store response by key
    return response
```

Interview explanation:

- Safe retries become possible even with client/network failures.

16.2 Outbox Flow Pseudocode

```
BEGIN TX
    insert into orders(...)
    insert into outbox(event_type='order.created', payload=...)
COMMIT

publisher worker:
    fetch unpublished outbox rows
    publish to broker
    mark published_at
```

Interview explanation:

- Prevents dual-write inconsistency between DB and broker.

16.3 Error Budget Math Example

Given SLO 99.9% monthly availability:

- Allowed error budget = 0.1%.
- For 1,000,000 requests/month, allowed errors = 1,000.

Interview explanation:

- This turns reliability discussion into measurable policy.

16.4 Burn Rate Alert Example

```
if short_window_burn_rate > 14 and long_window_burn_rate > 2:
    page on-call
```

Interview explanation:

- Multi-window burn rate catches fast and slow error budget consumption.

17) Distributed Systems and Hotstar-Style Streaming Drill

17.1 CAP/PACELC in Real Design Answers

- CAP for partition scenarios: choose consistency or availability per domain.
- PACELC for normal operation: latency vs consistency trade-off still exists.
- Payments, auth, entitlements often require stronger consistency than feed/recommendations.

17.2 Consistency Pattern Mapping

- Strong consistency: account balance, billing state.
- Read-your-writes: profile updates and user settings.
- Eventual consistency: recommendations, counters, popularity stats.

17.3 Hotstar/OTT Live Event Thought Process

HLD anchors:

- Edge/CDN first for global fan-out.
- Origin shielding to protect backend during spikes.
- Segment-based ABR streaming for variable network quality.

- Dedicated control plane for auth/entitlement/session.
- Event telemetry pipeline for QoE and anomaly detection.

Capacity framing example:

- Peak concurrent viewers and average bitrate determine egress requirement.
- Control-plane RPS and telemetry events are separate scaling dimensions.

17.4 Failure Modes to Call Out Explicitly

- Regional CDN degradation -> failover policy and blast-radius containment.
- Auth service timeout -> graceful degradation or limited fallback mode.
- Cache stampede on popular match start -> pre-warm + request coalescing.
- Message broker lag during spikes -> bounded retries and shed non-critical workloads.

17.5 L4 Grilling Questions

1. Where do you enforce entitlement checks to avoid token replay?
2. How do you protect origin when cache hit ratio drops suddenly?
3. Which components require multi-region active-active and why?
4. What SLOs do you define for startup time, rebuffer ratio, and error rate?